

Chapter 16. Acceleration Structures

So what are acceleration structures? In computer science terminology, when you try to rank every item in a corpus one by one, the typical amount of time it would take if there are N items is proportional to N . This is called big O notation. So if you have a user vector and you have a corpus of N items, it would take typically $O(N)$ time to score all the items in the corpus for one user. This is usually tractable if N is small and can fit into GPU RAM, typically $N < 1$ million items or so. However, if we have a very large corpus of, say, a billion items, it might take a very long time if we also have to make recommendations for a billion users. Then in big O notation it would be $O(10^{18})$ dot products to score a billion items for each and every one of a billion users.

In this chapter, we will try to reduce the $O(N * M)$ time to something sub-linear in the number of items N and the number of users M . We will discuss strategies including the following:

- Sharding
- Locality sensitive hashing
- k -d Trees
- Hierarchical k-means
- Cheaper retrieval methods

We'll also cover the trade-offs related to each strategy and what they could be used for. For all the following examples, we assume that the user and items are represented by embedding vectors of the same size and that the affinity between the user and items is a simple dot product, cosine distance, or Euclidean distance. If we were to use a neural network like a two-tower model to score the user and item, then possibly the only method that could be used to speed things up would be sharding or some kind of cheaper pre-filtering method.

Sharding

Sharding is probably the simplest strategy to [divide and conquer](#). Suppose you have k machines, N items, and M users. Using a sharding strategy, you can reduce the runtime to $O(N * M / k)$. You can do this by assigning each item a unique identifier, so you have tuples of (*unique_id*, *item_vector*). Then, by simply taking $\text{machine_id} = \text{unique_id} \% K$, we can assign a subset of the corpus to a different machine.

When a user needs a recommendation, we can then compute the top-scoring recommendations either ahead of time or on demand by distributing the workload onto k machines, thus making the computation k times faster, except for the overhead in gathering the top results on the server and ordering them jointly. Note that if you want, say, 100 top-scoring items, you would still have to obtain the top 100 results from each shard, collate them together, and then sort all the results jointly if you want to have the same results as in a brute-force method of scoring the entire corpus.

Sharding is useful in the sense that it can be combined with any of the other acceleration methods and is not dependent on the representation having any specific form, such as being a single vector.

Locality Sensitive Hashing

Locality sensitive hashing (LSH) is an interesting technique that converts a vector into a token-based representation. This is powerful because if CPUs are readily available, we can use them to compute the similarity between vectors by using cheaper integer arithmetic operations such as XOR and bit counting with specialized assembly instructions rather than floating-point operations. Integer operations tend to be much faster on CPUs than floating-point operations, so we can compute similarity between items much faster than using vector operations.

The other benefit is that once items are represented as a series of tokens, a regular search engine database would be able to store and retrieve

these items by using token matching. Regular hashing, on the other hand, tends to result in vastly different hash codes if a slight change occurs in the input. This is not a criticism of the hash functions; they just have different uses for different kinds of data.

Let's walk through a couple of ways to convert a vector into a hash. LSH is different from regular hashing in that small perturbations to a vector should result in the same hash bits as the hash of the original vector. This is an important property as it allows us to look up the neighborhood of a vector by using fast methods such as hash maps. One simple hashing method is called [the Power of Comparative Reasoning](#), or Winner Take All hashing. In this hashing scheme, the vector is first permuted using a known, reproducible permutation. We can generate this known permutation by simply shuffling the indices of all the vector dimensions with a random-number generator that accepts a seed and reliably reproduces the same exact shuffle sequence. It is important that the permutation is stable over different versions of Python, as we want to reproduce the hashing operation when generating the hashes as well as during retrieval time. Since we are using JAX's random library and JAX is careful about the reproducibility of permutations, we just directly use the permutation function in JAX. The hash code computation after that is simply a comparison between adjacent dimensions of the permuted vector, as shown in [Example 16-1](#).

Example 16-1. Winner take all

```
def compute_wta_hash(x):
    """Example code to compute some Winner take all hash vectors
    Args:
        x: a vector
    Result:
        hash: a hash code
    """
    key = jax.random.PRNGKey(1337)
    permuted = jax.random.permutation(key, x)

    hash1 = permuted[0] > permuted[1]
    hash2 = permuted[1] > permuted[2]
```

```

    return (hash1, hash2)

x1 = jnp.array([1, 2, 3])
x2 = jnp.array([1, 2.5, 3])
x3 = jnp.array([3, 2, 1])
x1_hash = compute_wta_hash(x1)
x2_hash = compute_wta_hash(x2)
x3_hash = compute_wta_hash(x3)
print(x1_hash)
print(x2_hash)
print(x3_hash)

(Array(False, dtype=bool), Array(True, dtype=bool))
(Array(False, dtype=bool), Array(True, dtype=bool))
(Array(True, dtype=bool), Array(False, dtype=bool))

```

As you can see, the vector `x2` is slightly different from `x1` and results in the same hash code of `01`, whereas `x3` is different and results in a hash code of `10`. The [Hamming distance](#) of the hash code is then used to compute the distance between two vectors, as shown in [Example 16-2](#). The distance is simply the XOR of the two hash codes, which results in 1 whenever the bits disagree, followed by bit counting.

Example 16-2. Hamming function

```

x = 16
y = 15
hamming_xy = int.bit_count(x ^ y)
print(hamming_xy)
5

```

Using the Hamming distance as shown here results in some speedup in the distance computation, but the major speedup will come from using the hash codes in a hash map. For example, we could break up the hash code into 8-bit chunks and store the corpus into shards keyed by each 8-bit chunk, which results in a 256× speedup because we have to look only in the hash map that has the same key as the query vector for nearest neighbors.

This has a drawback in terms of recall, though, because all 8 bits have to match in order for an item to be retrieved that matches the query vector. A tradeoff exists between the number of bits of the hash code used in hashing and the Hamming distance computation. The larger the number of bits, the faster the search, because the corpus is divided into smaller and smaller chunks. However, the drawback is that more and more bits have to match, and thus all the hash code bits in a nearby vector in the original space might not match and thus might not be retrieved.

The remedy is to have multiple hash codes with different random-number generators and repeat this process a few times with different random seeds. This extra step is left as an exercise for you.

Another common way to compute hash bits uses the [Johnson-Lindenstrauss lemma](#), which is a fancy way of saying that two vectors, when multiplied by the same random Gaussian matrix, tend to end up in a similar location. However, the L2 distances are preserved, which means this hash function works better when using Euclidean distance to train the embeddings rather than dot products. In this scheme, only the hash code computation differs; the Hamming distance treatment is exactly the same.

The speedup from LSH is directly proportional to the number of bits of the hash code that have to be an exact match. Suppose only 8 bits of the hash code are used in the hash map; then the speedup is 2^8 , or 256 times the original. The trade-off for the speed is having to store the hash map in memory.

k-d Trees

A common strategy for speeding up computation in computer science is *divide and conquer*. In this scheme, the data is recursively partitioned into two halves, and only the half that is relevant to the search query is searched. In contrast to a linear $O(n)$ in the number of items in the corpus scheme, a divide-and-conquer algorithm would be able to query a corpus in $O(\log_2(n))$ time, which is a substantial speedup if n is large.

One such binary tree for vector spaces is called a [k-d tree](#). Typically, to build a k -d tree, we compute the bounding box of all the points in the collection, find the longest edge of the bounding box and split it down the middle of that edge in the splitting dimension, and then partition the collection into two halves. If the median is used, the collection is more or less divided into two equal-numbered items; we say *more or less* because there might be ties along that split dimension. The recursive process stops when a small number of items is left in the leaf node. Many implementations of k -d trees exist—for example, [SciPy's k-d tree](#).

Although the speedup is substantial, this method tends to work when the number of feature dimensions of the vector is low. Also, similar to other methods, k -d trees work best when the L2 distance is the metric used for the embedding. Losses in retrieval can occur if the dot product was used for the similarity metric, as the k -d tree makes more sense for Euclidean space partitioning.

[Example 16-3](#) provides sample code for splitting a batch of points along the largest dimension.

Example 16-3. Partitioning via a k-d tree

```
import jax
import jax.numpy as jnp

def kdtree_partition(x: jnp.ndarray):
    """Finds the split plane and value for a batch of vectors x."""
    # First, find the bounding box.
    bbox_min = jnp.min(x, axis=0)
    bbox_max = jnp.max(x, axis=0)
    # Return the largest split dimension and value.
    diff = bbox_max - bbox_min
    split_dim = jnp.argmax(diff)
    split_value = 0.5 * (bbox_min[split_dim] + bbox_max[split_dim])
    return split_dim, split_value

key = jax.random.PRNGKey(42)
x = jax.random.normal(key, [256, 3]) * jnp.array([1, 3, 2])
split_dim, split_value = kdtree_partition(x)
print("Split dimension %d at value %f" % (split_dim, split_value))
```

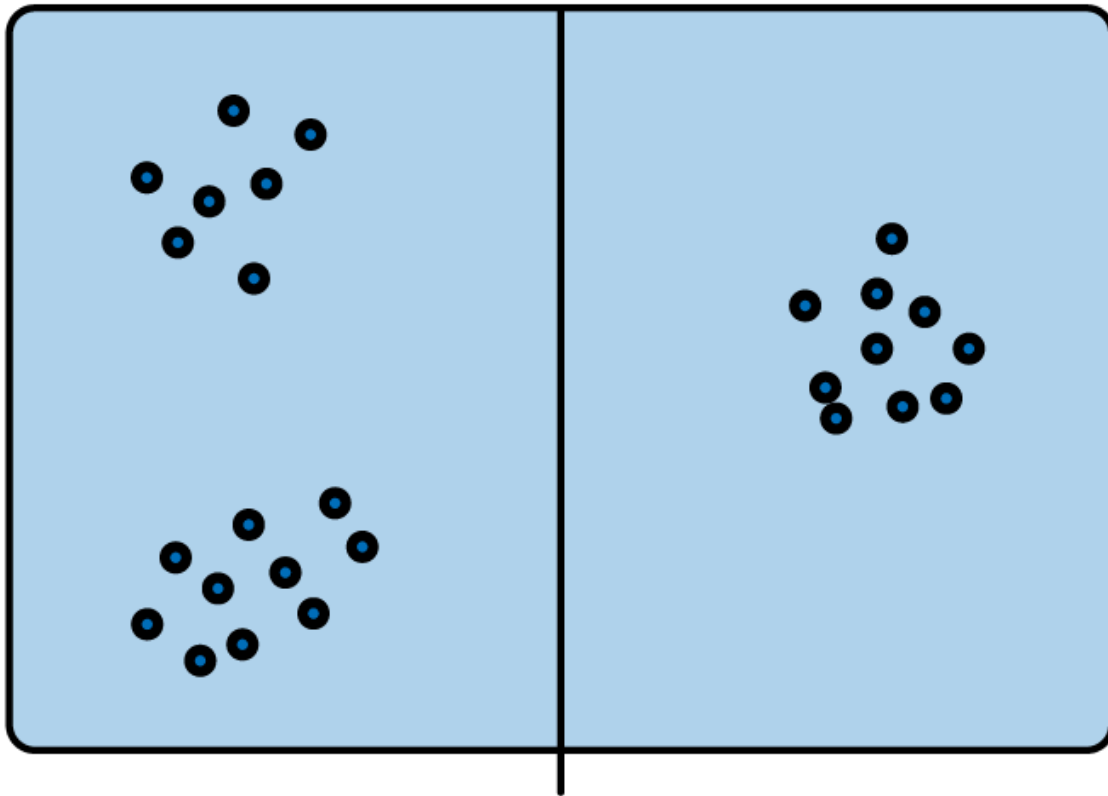
```
# Partition the points into two groups, the left subtree
# has all the elements left of the splitting plane.
left = jnp.where(x[:, split_dim] < split_value)
right = jnp.where(x[:, split_dim] >= split_value)

Split dimension 1 at value -0.352623
```

As you can see from the code, the k -d tree partitioning code can be as simple as splitting along the middle longest dimension. Other possibilities are splitting along the median of the longest dimension or [using a surface area heuristic](#).

A k -d tree is constructed by repeatedly partitioning the data along only one spatial dimension at a time (usually along the largest axis aligned to the spread of data); see [Figure 16-1](#).

Given a set of points, find a good splitting axis



Split along the largest dimension

Figure 16-1. k -d tree construction's initial bounding box

Partitions are recursively subdivided again, usually along the longest axis, until the number of points in the partition is fewer than a chosen small number; see [Figure 16-2](#).

The k -d tree lookup time is $O(\log_2(n))$ in n , the number of items in the corpus. The tree also requires a small overhead of memory to store the tree itself, which is dominated by the number of leaf nodes, so it would be best to have a minimal number of items in a leaf to prevent splits that are too fine.

Recursively partition and split until leaf nodes have a small number of items

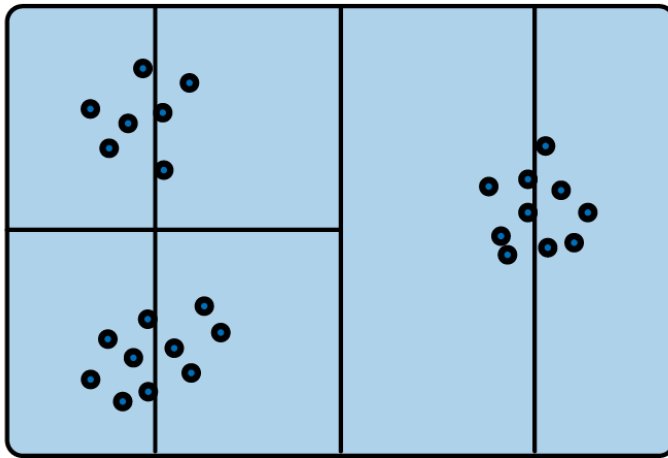


Figure 16-2. k-d tree construction recursively partitioned

From the root node, repeatedly check whether the query point (e.g., the item we are seeking nearest neighbors for) is in the left or right child of the root node, as shown in [Figure 16-3](#). For example, use `go_left = x[split_dim] < value_split[dim]`. In binary tree convention, the left child contains all points whose value at the split dimension are less than the split value. Hence if the query point's value at the split dimension is less than the split value we go left, otherwise we go right. Recursively descend down the tree until reaching the leaf node; then exhaustively compute distances to all items in the leaf node.

Number from the root node. See if the point is in the left or right child
i.e., $x[\text{split_dim}] < \text{value}[\text{split_dim}]$

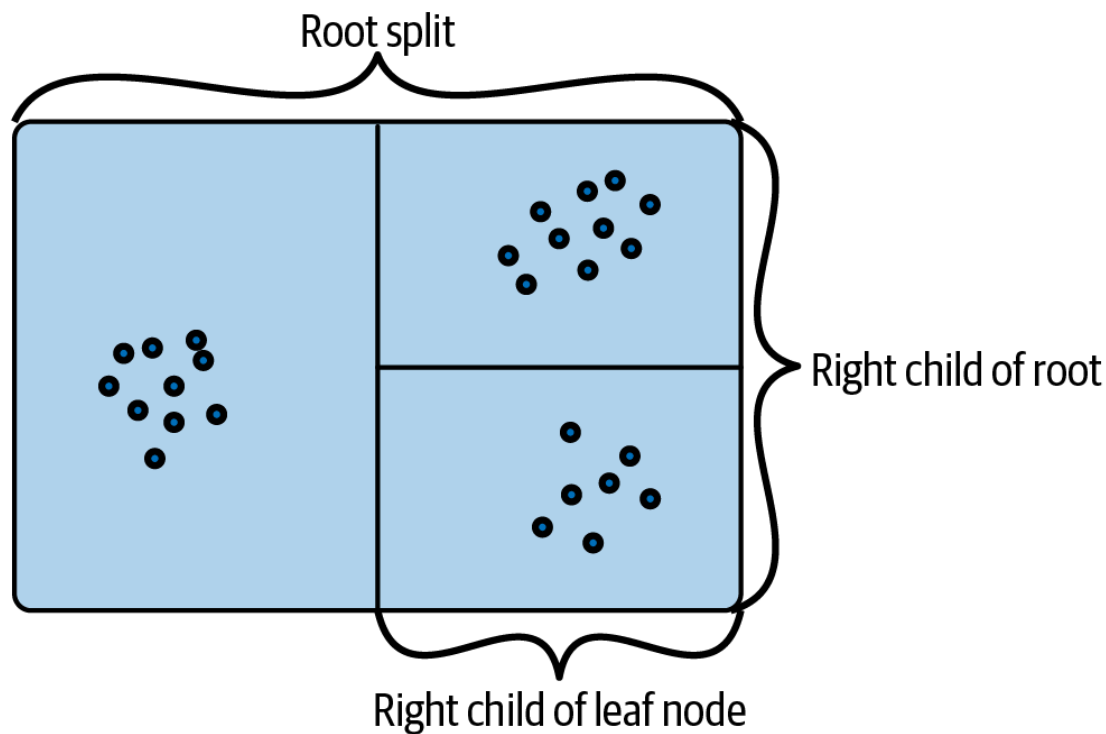


Figure 16-3. *k-d tree query*

A *k-d* tree has a potential drawback. If an item is close to a splitting plane, that item would be considered on the other side of the tree. As a result, the item would not be considered as a nearest neighbor candidate. In some implementations of *k-d* trees, called *spill trees*, both sides of a splitting plane are visited if the query point is close enough to the plane's decision boundary. This change increases runtime a little bit for the benefit of more recall.

Hierarchical k-means

Another divide-and-conquer strategy that does scale to higher feature dimensions is *k-means clustering*. In this scheme, the corpus is clustered into *k* clusters and then recursively clustered into *k* more clusters until each cluster is smaller than a defined limit.

An implementation of *k-means* can be found at [scikit-learn's web page](#).

To build the clustering, first create cluster centroids at random from existing points ([Figure 16-4](#)).

1. Assign points to nearest cluster

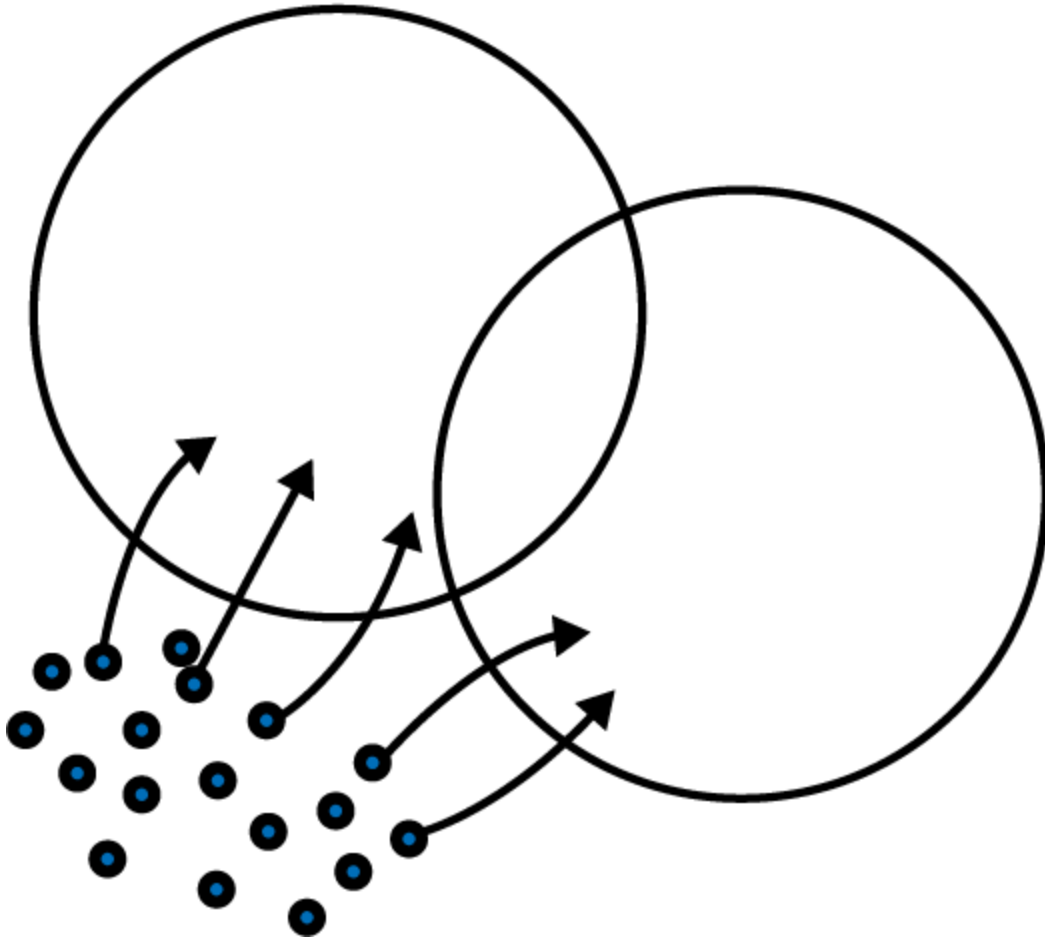
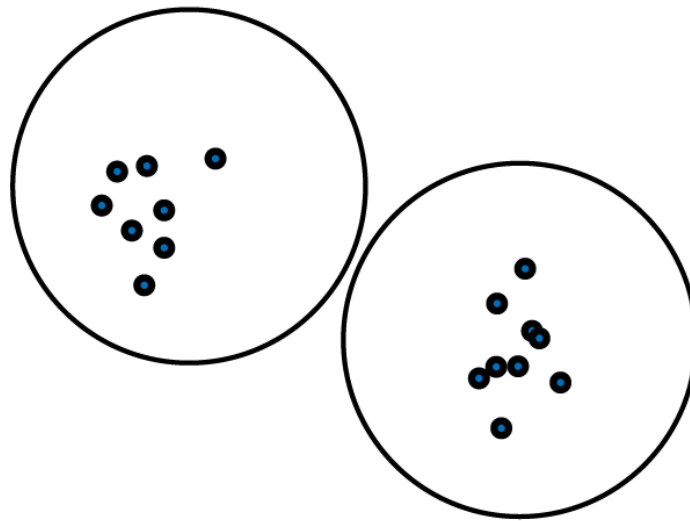


Figure 16-4. k-means initialization

Next, we assign all points to the cluster they are closest to. Then for each cluster, we take the average of all the assigned points as the new cluster center. We repeat until done, which can be a fixed number of steps.

[Figure 16-5](#) illustrates this process. The output is then k cluster centers of points. The process can be repeated again for each cluster center, splitting again into k more clusters.

2. Set new cluster center to average of all points assigned to cluster



3. Repeat until done

Figure 16-5. k-means clustering

Again, the speedup is $O(\log(n))$ in the number of items, but k -means is better adapted to clustering higher-dimensional data points than k -d trees.

The querying for a k -means cluster is rather straightforward. You can find the closest cluster to the query point and then repeat the process for all subclusters until a leaf node is found; then all the items in the leaf node are scored against the query point.

An alternative to k -means is to perform SVD and use the first k eigenvectors as the clustering criteria. The use of SVD is interesting in that there exists closed form and approximate methods like [power iteration](#) for computing the eigenvectors. Using the dot product to compute affinity might be better suited to vectors trained using the dot product as the affinity metric.

To learn more on this topic, you can consult [“Label Partitioning for Sublinear Ranking”](#) by Jason Weston et al. (including one of this book’s authors). The paper compares LSH, SVD, and hierarchical k -means. You’ll find a comparison of the speedup and the loss in retrieval, with the brute-force as a baseline.

An emerging trend in ANNs is using graph-based methods. Lately, *hierarchical navigable small worlds* is a particularly popular approach. This [graph algorithm](#) encodes proximity in multilayer structures and then relies on the common maxim that “the number of connectivity steps from one node to another is often surprisingly small.” In graph-based ANN methods, you often find one neighbor, and then traverse the edges connected to that neighbor to rapidly find others.

Cheaper Retrieval Methods

If your corpus has the ability to do an item-wise cheap retrieval method, one way to speed up searches is to use the cheap retrieval method to obtain a small subset of items and then use the more expensive vector-based methods to rank the subset. One such cheap retrieval method is to make a posting list of the top co-occurrences of one item with another. Then when it comes to generating the candidates for ranking, gather all the top co-occurring items together (from a user’s preferred items, for example) and then score them together with the ML model. In this way, we do not have to score the entire corpus with the ML model but just a small subset.

Summary

In this chapter, we showed a few ways to speed up the retrieval and scoring of items in a corpus, given a query vector, without losing too much in terms of recall and while still maintaining precision. No ANN method is perfect, as the acceleration structures depend on the distribution of the data, and this varies from dataset to dataset. We hope that this chapter provides a launching pad for you to explore various ways to make retrieval faster and sublinear in the number of items in the corpus.